B.Sc. (Hons) in Software Development

Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

# AI Teaching Assistant - AITA Website

https://github.com/PhatTanNg/AI-Teaching-Assistance-Project

**By**
**Phat Tan Nguyen**

April 25, 2026

## Minor Dissertation

**Department of Computer Science & Applied Physics,**
**School of Science & Computing,**
**Atlantic Technological University (ATU), Galway.**

# Contents


**1 Introduction** **2**
1.1 Background and Motivation . . . 2
1.2 Problem Statement . . . 3
1.3 Project Aim . . . 4
1.4 Objectives . . . 4
1.5 Implemented Scope and Boundaries . . . 4
1.6 Significance of the Work . . . 5
1.6.1 Educational Significance . . . 5
1.6.2 Technical Significance . . . 6
1.7 Proposal-to-Implementation Evolution . . . 6
1.8 Methodological Positioning . . . 7
1.9 Research Questions . . . 7
1.10 Dissertation Structure . . . 7
1.11 Chapter Summary . . . 8

**2 Methodology** **9**
2.1 Chapter Overview . . . 9
2.2 Software Development Methodology . . . 9
2.2.1 Rationale for Method Selection . . . 9
2.2.2 Development Strategy . . . 10
2.2.3 Phased Implementation . . . 10
2.2.4 Planning and Requirements Elicitation . . . 11
2.2.5 Proposal-to-Implementation Adaptation . . . 11
2.2.6 Meetings, Feedback, and Governance . . . 11
2.3 Research and Evaluation Methodology . . . 12
2.3.1 Methodological Position . . . 12
2.3.2 Evaluation Framework . . . 12
2.3.3 Validation and Testing Approach . . . 12
2.3.4 Empirical Problem-Solving Procedure . . . 13
2.3.5 Data and Evidence Sources . . . 13
2.4 Threats to Validity . . . 14
2.4.1 Internal Validity . . . 14
2.4.2 External Validity . . . 14
2.4.3 Construct Validity . . . 14
2.5 Ethical and Privacy Considerations . . . 14
2.6 Why This Methodology Is Valid for AITA . . . 14
2.7 Chapter Summary . . . 15

**3 Technology Review** **16**
3.1 Chapter Overview . . . 16
3.2 Review Criteria and Selection Logic . . . 16

3.4 Technology Context in AI-Assisted Learning Systems . . . . . 17
3.5 Frontend Technology Review . . . . . 18
3.5.1 React and Component-Based UI Architecture . . . . . 18
3.5.2 Vite Build Tooling and Development Throughput . . . . . 18
3.5.3 Routing, UI Consistency, and Internationalization . . . . . 18
3.5.4 State Management and Contextual Consistency . . . . . 19
3.5.5 Frontend Trade-offs and Alternatives . . . . . 19
3.6 Backend and API Technology Review . . . . . 20
3.6.1 Node.js and Express for Modular Service Delivery . . . . . 20
3.6.2 SSE and Request-Response Hybrid Communication . . . . . 20
3.6.3 Authentication and Account Flows . . . . . 20
3.6.4 API Boundary Design and Module Cohesion . . . . . 20
3.6.5 Backend Technology Alternatives . . . . . 21
3.7 Data Layer Technology Review . . . . . 21
3.7.1 MongoDB for Flexible Educational Content Storage . . . . . 21
3.7.2 Mongoose for Schema Discipline in an Evolving System . . . 21
3.7.3 Data Modeling Strategy for Learning Artifacts . . . . . 22
3.7.4 Consistency, Indexing, and Query Practicality . . . . . 22
3.8 AI and NLP Technology Review . . . . . 22
3.8.1 Speech-to-Text and Transcript Processing . . . . . 22
3.8.2 Generative AI for Correction, Keyword Support, and Study Artifacts . . . . . 23
3.8.3 Model Capability vs. System Responsibility . . . . . 23
3.8.4 Speech Technology Practicality and Platform Constraints . . 23
3.8.5 Python Microservice Role and Pragmatic NLP Choices . . . 23
3.9 Deployment and Operations Technology Review . . . . . 24
3.9.1 Service Split Across Vercel and Render . . . . . 24
3.9.2 Configuration and Environment Separation . . . . . 24
3.9.3 Operational Trade-offs . . . . . 24
3.9.4 Operational Evidence from Integration Fixes . . . . . 25
3.10 GitHub Evidence of Technology Evolution . . . . . 25
3.11 Security and Privacy Technology Review . . . . . 26
3.12 Testing and Quality-Engineering Technology Review . . . . . 26
3.13 Synthesis: Objective-to-Technology Alignment . . . . . 27
3.14 Comparison with Alternative Approaches . . . . . 28
3.15 Technology Limitations and Trade-offs . . . . . 28
3.16 Why This Technology Stack Is Appropriate for AITA . . . . . 29
3.17 Chapter Summary . . . . . 29

**4 System Design** **30**
4.1 Chapter Overview . . . 30
4.2 System Architecture Design . . . 30
4.2.1 Architectural Style . . . 30
4.2.2 High-Level Architecture Diagram . . . 31
4.2.3 Design Rationale for Split Services . . . 31
4.3 UML-Oriented Design Views . . . 31
4.3.1 Use-Case View . . . 31
4.3.2 Component and Interaction View . . . 33
4.3.3 Domain Model View . . . 33
4.4 Backend System Design . . . 34
4.4.1 Route and Module Decomposition . . . 34
4.4.2 Server Composition Snippet . . . 35
4.4.3 Streaming Chat Design (SSE) . . . 35
4.4.4 Speech Stream Proxy Design . . . 36
4.5 Data Design and Persistence Strategy . . . 36
4.5.1 Core Entities . . . 36
4.5.2 Design Constraints Applied . . . 36
4.6 Frontend System Design . . . 37
4.6.1 Page and Component Architecture . . . 37
4.6.2 State and Context Design . . . 37
4.6.3 UI Evidence Screenshots . . . 37
4.7 Security-Aware Design . . . 37
4.7.1 Authentication and Access Control . . . 37
4.7.2 Design Trade-offs . . . 38
4.8 Core Algorithms and Processing Logic . . . 39
4.8.1 Algorithm 1: Transcript Processing Pipeline . . . 39
4.8.2 Algorithm 2: Context-Aware Chat Retrieval and Response . 41
4.9 Representative Implementation Snippet . . . 42
4.10 Design Decisions and Their Effects . . . 42
4.11 Scalability and Extensibility Considerations . . . 42
4.12 Chapter Summary . . . 43

**5 System Evaluation** **44**
5.1 Chapter Overview . . . 44
5.2 Evaluation Strategy . . . 44
5.2.1 Evaluation Basis . . . 44
5.2.2 Assessment Scale . . . 45
5.3 Objective-Based Evaluation . . . 45
5.4 Results from Real-World Feature Testing . . . 45
5.5 System Strengths . . . 45

5.5.1 Integrated End-to-End Workflow . . . . . 45
5.5.2 Practical Educational Utility . . . . . 46
5.5.3 Implementation Maturity for Final-Year Scope . . . . . 47
5.6 System Weaknesses and Limitations . . . . . 47
5.6.1 AI Output Variability . . . . . 47
5.6.2 Limited Automated Test Depth . . . . . 47
5.6.3 Evaluation Scope Boundary . . . . . 47
5.6.4 Operational Constraints . . . . . 47
5.7 Critical Reflection . . . . . 47
5.7.1 What Was Proven . . . . . 47
5.7.2 What Was Not Proven . . . . . 48
5.7.3 Implications for Future Iteration . . . . . 48
5.8 Threats to Evaluation Validity . . . . . 48
5.8.1 Internal Validity . . . . . 48
5.8.2 External Validity . . . . . 48
5.8.3 Construct Validity . . . . . 49
5.9 Chapter Summary . . . . . 49

**6 Conclusion** **51**
6.1 Chapter Overview . . . . . 51
6.2 Project Context and Rationale Recap . . . . . 51
6.3 Objectives Recap . . . . . 52
6.4 Summary of Evaluation Findings . . . . . 52
6.4.1 What the Project Successfully Delivered . . . . . 52
6.4.2 What Remains Limited . . . . . 53
6.5 Overall Contribution . . . . . 53
6.6 Critical Final Reflection . . . . . 54
6.7 Future Work Directions . . . . . 54
6.8 Final Closing Statement . . . . . 54

**A Source Code Repository** **56**

# List of Figures


4.1 AITA high-level system architecture. . . . . 32
4.2 End-to-end workflow from lecture ingestion to study outputs. . . . . 33
4.3 Conceptual data model of user, lecture, transcript, and revision artifacts. . . . . 34
4.4 Home page interface and entry workflow. . . . . 38
4.5 Transcription interface (live/upload flow and transcript workspace). 38
4.6 Saved transcripts management interface. . . . . 39
4.7 Keyword and definition review interface. . . . . 39
4.8 Mind map generation and visualization interface. . . . . 40
4.9 Profile and account settings interface. . . . . 40
4.10 Authentication and security flow design. . . . . 41

# List of Tables

3.1 Representative GitHub evidence for technology evolution . . . . . 26
3.2 Objective-to-technology alignment in AITA . . . . . 27

4.1 Design decision and impact mapping. . . . . 43

5.1 Evaluation of AITA against project objectives . . . . . 46
5.2 Representative real-world feature test cases . . . . . 50

# Chapter 1

# Introduction

## 1.1 Background and Motivation

Fast-paced lecturing in higher education is becoming more and more a fact, particularly in interdisciplinary and technical classes. This format is effective in covering content, but poses a continuous learner-side obstacle: students have to listen, comprehend, spot key terminologies and write notes which are useful within time constraints. Practically, this usually results in the disjointed quality of notes, lack of conceptual connection and poor revision readiness after the lecture. The students might walk out of the classroom having attended a session but lack material to study which is well organized to be retained over the long run.

This is the gap between exposure during lectures and the understanding that is revision ready and this is the driving force behind this project. Existing note-taking devices aid storage but do not typically actively transform the content. That is, any workflow that is presently available stores information without helping to address cognitive overheads substantially. Students continue to do manual cleaning, dictionary searching and reorganizing until they can start to revise meaningfully.

Recent developments on web engineering and language processing with the aid of AI give a viable path to minimize this load. New client-server systems are able to carry the enduring and customized workflows across gadgets [1, 2, 3, 4, 5, 6]. Similarly, speech-to-text and generation APIs can automatically convert lectures audio into structured learning materials [7, 8, 9, 10]. This technical environment aids in the creation of systems that capture the content, and make it ready to be actively learned.

In this setting is the AITA (AI Teaching Assistance) project. It is an end-to-end web platform, which transforms lecture inputs into useful learning outputs. The project combines capture, processing, comprehension support workflows and

revision workflows into a single coherent system as opposed to having a single isolated feature.

## 1.2 Problem Statement

The issue covered by this dissertation could be as follows: What is needed is how a student facing system can minimize time and mental effort to convert raw lecture delivery into quality, revision oriented study material and still remain usable, secure and ready to be deployed in practice?

This issue has both educational and technical aspects. Educationally, there are three barriers that recurrently occur in students:

1. Real-time capture pressure: one is divided between the listening and writing notes.

2. Conceptual interruption: new words compel outside searching, disrupting the flow of lectures.

3. Revision friction: it is hard to transform unstructured notes into active-recall materials.

Technically, this issue cannot be resolved by just transcription. It involves a pipeline that is able to:

- Read aloud in real-life study situations,
- Improve readability of raw transcript output,
- Identify and describe important words,
- Generate revision artifacts,
- Store user information safely to be accessed again,
- Have a non-technical user-friendly interface.

This makes novelty within a single component of the algorithm not the goal of the project, but instead provide real-world utility at the system level through the entire student workflow.

## 1.3 Project Aim

The objective of the project is to design, implement and test a full-stack AI-assisted lecture learning system that turns lecture speech into reusable, structured and revision-friendly educational materials.

The platform must facilitate the active study process over passive attendance, which is integration of transcription, term support, synthesis of content and revision interaction within one web application.

## 1.4 Objectives

In order to achieve this goal, the following objectives are determined in the project:

1. Offer quality lecture capture services of live speech and audio upload.
2. Enhance the readability of transcripts with the help of AI.
3. Find significant academic keywords and give a brief explanation of terms.
4. Create useful revision products (summary, mind map, flashcards and MCQs) based on lecture materials saved.
5. Facilitate active memory and interaction by means of interactive revision processes.
6. Produce a working online application with the end-to-end data that is saved within the user-specific context.
7. Make sure to have secure account and content handling using authenticated access control.
8. Show depth in final year project in terms of integrated architecture, deployable services and multi-module delivery.

These objectives are later reused as explicit evaluation anchors in Chapter 5, ensuring methodological traceability between design intent and measured outcome.

## 1.5 Implemented Scope and Boundaries

The most important need in this dissertation is that it must be aligned with implemented functionality, as opposed to assumptions in proposals. The implemented scope entails:

- Password reset, email verification, user registration, and sign-in.
- Live and file lecture transcription processes.
- Transcript readability correction.
- Extraction and explanation of key words.
- Summary generation.
- Mind map generation.
- MCQ and flashcard generation for revision.
- Contextual AI chat assistance related to lectures.
- Bilingual interface support (English and Vietnamese).
- Continuous backing up of user content and revision information.
- Frontend and backend services that can be deployed to the cloud.

Out-of-scope features are clearly not part of an implementation assertion, such as notebook/document aggregation extensions that are planned to be described as future expansion concepts [11]. This definition of the boundary does not over-claim and retains the academic reliability.

## 1.6 Significance of the Work

### 1.6.1 Educational Significance

The aim of AITA is to minimize the post-lecture processing overhead and enhance readiness to revise. The system automates important transformation steps instead of having to manually convert raw notes into active study material. This is consistent with data that retrieval-based study behavior has the potential to enhance learning compared to simply passive review methods [12].

The real effect is most potent in lectures with heavy terminology and rapid information flow where students are likely to suffer incompleteness of notes and loss of concepts.

### 1.6.2 Technical Significance

The project exhibits integration of:

- Modern frontend architecture,
- Backend service design,
- Continuing educational data modeling,
- AI-assisted transformation pipelines,
- Deployment-ready multi-service configuration.

This contributes to the fact that the project is appropriate to be evaluated in final-year: it is based on real-world engineering trade-offs in integration, reliability, and maintainability, as opposed to a single isolated prototype feature [13, 14].

## 1.7 Proposal-to-Implementation Evolution

The project had an initial proposal context and was developed through the process of making decisions on implementation. This development is anticipated in practice software undertakings and is addressed as a methodological asset as opposed to a deviation.

In development, the decisions were narrowed down on the basis of:

- Deployment compatibility,
- Runtime reliability,
- Integration complexity,
- User workflow coherence,
- Feature-level implementation within final-year constraints.

Consequently, the final system will be an evidence-based implementation path that is more practical but maintains the initial educational goal [11].

## 1.8 Methodological Positioning

This dissertation is placed as a systems project, which is implementation-based and has objectives in its evaluation. It does not purport to a new underpinning NLP architecture. Rather it is an architectural and functional contribution:

1. Constructing a sensible lecture-to-revision pipeline,
2. Combining various AI-based applications into a single platform,
3. Proving feasibility by functionality delivered and implemented.

This is a suitable positioning in case of undergraduate applied project assessment where integration quality and practical value are the main criteria.

## 1.9 Research Questions

In order to divide the dissertation, the following guiding questions are employed:

- **RQ1**: Does an AI-assisted web platform with an integrated workflow help lessen the friction between lecture capture and revision preparation?
- **RQ2**: Does the system provide reliable end-to-end functionality in terms of transcription, comprehension support and revision generation?
- **RQ3**: What are the quality, robustness and empirical educational validation limitations post-implementation?

The answers to these questions are discussed in the case of system design evidence and evaluation based on objectives in subsequent chapters.

## 1.10 Dissertation Structure

The rest of this dissertation will be structured in the following way:

- **Chapter 2 (Methodology)** describes development strategy, stages of implementation, validation strategy and proposal-to-implementation adaptation logic.
- **Chapter 3 (Technology Review)** provides a review of the chosen technologies and reasons why they are relevant to this problem area.

- **Chapter 4 (System Design)** provides evidence on architecture, modules, data model, workflow, and security.
- **Chapter 5 (System Evaluation)** evaluates the provided system in terms of the established objectives and addresses the strengths and weaknesses.
- **Chapter 6 (Conclusion)** summarizes the contributions, reflects on the impact of implementations and plans further extensions.

## 1.11 Chapter Summary

This chapter presented the teaching and technical background of the AITA project, formulated the problem statement, established the objectives, defined the scope limits, and placed the research within the dissertation structure which is focused on implementation. It also gave the importance of the project and gave the framework of the roadmap of the entire report.

The following chapter explains the procedure followed to construct and test the system in practice.

# Chapter 2

# Methodology

## 2.1 Chapter Overview

This chapter describes the planning, implementation and validation of the AITA project. Two complementary perspectives are given on the methodology:

1. Software development methodology: the way the system was designed to go to a deployable product.
2. Research and evaluation methodology: how the implemented system was reviewed in comparison to the objectives of the project.

The main methodological argument is that an iterative, implementation-based approach was justifiable in this project as the issue needed to be addressed through the gradual, progressive optimisation of a number of interdependent modules (frontend, backend, AI orchestration, and database) instead of a single algorithm.

## 2.2 Software Development Methodology

### 2.2.1 Rationale for Method Selection

AITA has been created in the incremental and iterative manner of Agile. This decision was suitable due to three reasons:

- The system is made up of several components that are coupled (speech ingestion, transcript processing, keyword support, revision generation, account security).
- External AI services create variability in quality of output and latency, and necessitate repetitive integration checks.

- Educational workflows that are shown to the user must be refined through progressive usability.

This would have been riskier using a strict linear (waterfall-style) process, where architecture and service decisions had to be re-evaluated during implementation. The iterative developing process enabled safe adaptation without losing the initial learning purpose.

### 2.2.2 Development Strategy

The project was constructed in phases of feature delivery. All the phases were repeating:

1. Identify a feature objective.
2. Implement a minimal working version.
3. Test integrated behavior.
4. Refine for reliability and usability.
5. Lock scope and continue to the next module.

The cycle was implemented across all their key capabilities: transcription, correction, keyword support, summary, revision artifacts, mind map generation and contextual chat.

### 2.2.3 Phased Implementation

Implementation was based on repository and commit evolution, and developed as follows:

1. **Foundation phase**: project organisation, base frontend/backend configuration and first route integration.
2. **Speech pipeline phase**: ingestion and transcription workflow stabilisation of lectures.
3. **Keyword and comprehension phase**: integration and resilience extraction/definition path.
4. **Revision phase**: flashcards and MCQs creation, revision plans, and progress maps.

5. **Interaction phase**: support of contextual AI chat and multilingual interface.
6. **Visualisation and polish phase**: mind map creation, UI/UX refinement, and deployability hardening.

### 2.2.4 Planning and Requirements Elicitation

The main problem that was used to derive requirements was the learner workflow problem:

- Students should record lecture speech.
- Students require more explicit after-lecture text.
- Domain terminology must be supported in students.
- Students require revision artifacts in order to recall.

The requirements were converted to a learning pipeline end to end, as opposed to designing features in a vacuum. The scope was managed with priority on the implemented value as opposed to the speculative extensions.

### 2.2.5 Proposal-to-Implementation Adaptation

An explicit adaptation of project proposal assumptions to implementation reality is a strength of this project in terms of methodology. Service and design decisions were redefined throughout the development process, according to the integration constraints, platform suitability, and practical reliability.

These modifications are not considered uncontrolled scope drift, but a controlled engineering decision. The educational goal was not changed; only the implementation details changed.

### 2.2.6 Meetings, Feedback, and Governance

The practice of project governance was a regular review-and-refinement cycle.

- Frequency of meetings between supervisor and student: once a week.
- Form of review: progress update, blockers, feature demo, next milestones.
- Feedback options: supervisor comments and self-review of implementation between checkpoints on a weekly basis.

Periodic review checkpoints are significant even in development where most of it is individual in nature, to ensure scope control and mitigation of risk.

## 2.3 Research and Evaluation Methodology

### 2.3.1 Methodological Position

The dissertation uses the applied systems evaluation methodology. The contribution is not presented as a new underlying machine-learning algorithm; rather it is the design, implementation, and goal-oriented evaluation of a combined AI-assisted learning system.

Based on this, assessment is organised about whether the functionality implemented meets pre-determined goals under realistic operational environments.

### 2.3.2 Evaluation Framework

Evaluation of the project is based on objectives:

- O1: Lecture capture and transcription reliability.
- O2: Transcript readability improvement.
- O3: Keyword and definition support.
- O4: Generation of revision artifacts.
- O5: Active-recall workflow support.
- O6: End-to-end usability.
- O7: Account and content security.
- O8: Full-stack integration depth.

Chapter 5 evaluates each of the objectives with implementation evidence and identified limitations.

### 2.3.3 Validation and Testing Approach

The validation was performed on various levels:

1. **Functional validation**: correctness of endpoint behavior and features.
2. **Integration verification**: full module-to-module user journeys.
3. **Service verification**: AI-conditioned flows and fallback.
4. **Error-path checking**: bad input and edge-case checking.

Backend health and analysis checks were done with a Python service test script, and the majority of frontend/backend flows were tested with practical integration testing.

Besides the in-house validation, the project had approximately ten real-world feature-based user test cases. These were test cases to confirm functional behavior and usability flow (e.g., transcription and generated outputs and revision interactions) but not a complete production deployment or a long-term rollout to classrooms.

*Known limitation*: extensive automated test coverage (unit/component/e2e) is not comprehensive and is specified as future work.

### 2.3.4 Empirical Problem-Solving Procedure

The project followed an empirical debug-refine loop:

1. Observe issue in live workflow.
2. Isolate failure source.
3. Patch service/UI/model logic.
4. Re-run integrated scenario.

This approach was repeatedly applied to stabilize multi-stage flows, especially where AI output variability affected downstream processing.

### 2.3.5 Data and Evidence Sources

In this dissertation, methodological evidence is based on:

- Implemented repository structure and modules,
- Route, service, and model behavior,
- Deployment configuration,
- Phased commit history,
- System behavior and artifacts of validation.

This is evidence-based and does not make assumptions which are proposal-only.

## 2.4 Threats to Validity

### 2.4.1 Internal Validity

The side effects of integration between modules and variability of AI output are possible threats. Mitigation encompasses modular architecture, incremental integration and end-to-end checks.

### 2.4.2 External Validity

The results can best be generalised to the lecture-based study environment akin to the project case. Generalisation to broader educational settings requires larger user studies across institutions and disciplines.

### 2.4.3 Construct Validity

Implemented workflow capabilities infer some of the educational outcomes (e.g., long-term retention gain) instead of being confirmed by longitudinal controlled studies. This has been recognised as a limitation and an area for future research.

## 2.5 Ethical and Privacy Considerations

Since the system can handle user-created lecture material and account information, privacy-conscious design principles are taken into consideration:

- Authorised access to information owned by a user.
- Account verification and password-reset controls.
- Selective access to personal research materials.

No institutional compliance certification statements are made other than the application-level controls in place.

## 2.6 Why This Methodology Is Valid for AITA

The selected methodology is valid for this project because it:

1. Aligns with the system's multi-module complexity.
2. Supports controlled adaptation from proposal to implementation.

3. Produces a deployable artifact rather than isolated prototypes.
4. Enables objective-based evaluation tied to delivered features.
5. Clearly documents limitations and future work.

## 2.7 Chapter Summary

This chapter has presented the development and research methodology used in AITA. The project employed an iterative engineering process with objective-based evaluation. The approach enabled practical delivery of an integrated AI-assisted learning platform while preserving methodological traceability and transparency about current limitations. The next chapter reviews the key technologies and standards that informed implementation decisions.

# Chapter 3

# Technology Review

## 3.1 Chapter Overview

In this chapter, the technologies employed in AITA will be reviewed and the reasons as to why each of the choices was appropriate to meet the project objectives outlined in Chapter 1 will be justified. It is not a tool listing review. Rather, it connects the technology choices to the real issue in the educational workflow the conversion of lecture input to output that is ready to be revised and has acceptable usability, reliability and deployment feasibility.

This review is a synthesis of two sources of evidence in line with the implementation-centered approach in Chapter 2:

1. Technical literature and official documentation for architecture, APIs, and engineering practices.

2. Repository and GitHub evolution evidence showing how technology choices were validated and refined during real implementation.

## 3.2 Review Criteria and Selection Logic

Objective fit was used in choosing technology as opposed to novelty. All the tools were compared with four criteria:

- Educational workflow fit: Lecture capture, processing and generation of revision.

- Integration compatibility: capable of being used in a full-stack pipeline without being unduly glued together.

- Practicability: can be completed within final-year time limits and deliverable.

- Maintainability: for incremental upgrading, readability and modularity.

These criteria are consistent with O1 through O8 and will make technology choices accountable to the project results as opposed to subjectivity.

## 3.3 Technology Review Method and Evidence Scope

The review approach taken in this chapter adheres to an evidence synthesis approach which is practical. Rather than dividing the stories of related work and technology choices into two isolated stories, AITA unites them into a decision-based analysis:

- Define the technical requirement connoted by project goals,
- Read pertinent platform/documentation and current evidence of AI-in-education,
- Consider other avenues of implementation,
- Reason why the option chosen is done with the use of results of the repositories and deployment.

The approach fits an implementation dissertation since the focus of contribution is quality of integration within real-world constraints as opposed to novelty of algorithms. It enhances traceability as well: every technology that has been reviewed can be traced to actual modules, routes, and features that are delivered to the user in the repository [11].

## 3.4 Technology Context in AI-Assisted Learning Systems

Modern AI-supported learning models are typically tested against two concomitant aspects: teaching utility and technical dependability. The AI in education literature shows that to enable the application in the classroom, practical deployment is not just about the ability of the model, but also about the user-friendliness of the interface, continuity of the workflow, and control over quality when the model is used under the variability of real-world conditions [15]. In the case of AITA, this observation is key due to the presence of cognitively challenging conditions on the part of the students (listening, interpreting and organizing information on the spot).

Moreover, retrieval-practice studies justify the usefulness of transforming passive lecture presentation to active revision tests like flashcards and quizzes [12].

Thus, the technology assessment should determine the capabilities of the chosen tools to facilitate a full transformation chain: capture speech and transcript cleaning, conceptual support and artifact generation through active-recall.

On the systems side, the current web engineering stacks are becoming more popular as the choice of student-facing educational tools because of its accessibility, the short development cycle, and the possibility to deploy it to the cloud [1, 2, 3, 4]. AITA does so, and AI orchestration is introduced to minimize the amount of manual work to do after lectures.

## 3.5 Frontend Technology Review

### 3.5.1 React and Component-Based UI Architecture

React is the frontend basis at AITA [1]. The component model of React is suitable in this project since the interface comprises of reusable and stateful modules (transcription view, key word interactions, revision widgets and contextual chat surfaces). In a system with re-use of workflows in education, where new workflows can be reused, composable UI logic minimizes duplication, and enables more rapid iterative development.

Literature and practice Component-based architecture enhances separation of concerns and provides incremental improvement and enhancement in consumer facing systems. This can be seen in AITA by the modular page/component architecture and the support to add and develop features like profile flows, revision screens, and multilingual content, without having to re-write an entire page.

### 3.5.2 Vite Build Tooling and Development Throughput

Vite was chosen to develop and tooling to build [2]. The major benefit to AITA is the quick feedback in the process of UI iteration, particularly in the case of transcription UX, revision interactions, and language toggling. Faster startup and hot-update cycles are very beneficial in an iterative methodology, because they significantly decrease the amount of friction in debugging and polishing the usability.

This option helps the development loop in Chapter 2 (implement, test, refine) by reducing the amount of time to go through the code change and validation.

### 3.5.3 Routing, UI Consistency, and Internationalization

Modern frontend patterns (single-page routing and shared UI primitives) were used to implement routing and the organization of interfaces and bilingualism was

implemented as a first-class feature in line with the learner context.

History of i18n coverage and UI stability shows long-term investments in i18n coverage and cross-page polish. This further fortifies the argument that not only was frontend technology selected, but also operationalized to allow accessibility and usability results.

### 3.5.4 State Management and Contextual Consistency

The frontend architecture integrates local state in components with common context providers of cross-page concerns (such as authentication state, language choice and chat context). This is correct since AITA demands habitual actions in a variety of perspectives within a session: customers switch between transcription, transcript review, key-word research and revision without losing identity or workflow.

Page-isolated style would enhance synchronization bugs, and poor user experience. In comparison, context-based sharing makes the issues of a session explicit and the maintainability better in the context of the feature expansion during an iterative process. The design is following the guidance of React on the use of composable states and context in medium-scale apps [1].

### 3.5.5 Frontend Trade-offs and Alternatives

Other frontend realizations were open to it, such as page transitions being rendered by a server, or more heavyweight framework abstractions. In this project, the following were alternative alternatives whose value added with respect to complexity was minimal:

- Multipage architecture based on server rendering would make workflows which are transcription based less immediately interactive,
- An additional layer of abstraction would add greater overhead to set up without obvious benefits in end-year scope,
- Only custom UIs with no reusable primitives would be slower in accessibility and consistency enhancements.

React + Vite and reusable UI patterns is therefore a compromise to velocity, modularity and refinement of usability.

## 3.6 Backend and API Technology Review

### 3.6.1 Node.js and Express for Modular Service Delivery

AITA is based on Node.js and Express to coordinate API [3, 4]. The stack is appropriate due to the large number of endpoints to be coordinated (auth, transcript management, transcribe/correction flows, chat streaming, revision generation, and mind map support) that the project needs.

Express offers lightweight routing and middleware patterns sufficiently sizable to be used in a student project and yet, accommodating clean module boundaries. In the case of AITA, this allowed separation of controllers, routes and services and expansion in an iterative fashion, without disruptive rewrites.

### 3.6.2 SSE and Request-Response Hybrid Communication

The system integrates both traditional request-response API and streaming in AI chat. This hybrid system fits the interactions of educational assistants very well, where responsiveness influences the trust and interaction with the user. The chosen backend stack can enable this combination without the need to add to the infrastructure complexity that is not necessary.

### 3.6.3 Authentication and Account Flows

Authentication is applied to the patterns of JWT and account lifecycle (sign-up/sign-in, verification, password reset). This offers a realistic security/implementation-complexity tradeoff on project scope. The design enforces user-isolated data and compliant with privacy expectations with study artifacts.

### 3.6.4 API Boundary Design and Module Cohesion

The structure of the backend route represents domain oriented boundaries: authentication, content management, transcription services, chat and mind map generation and revision sub modules. This boundary design enhances cohesion since the group of routes is associated with a capability that is visible to the user and not an arbitrary technical level.

In the case of a multi-feature educational platform, it is significant to have this route-level modularity. It facilitates gradual feature delivery (as specified in Chapter 2), minimizes unwanted coupling and minimizes regression risk with one subsystem is changing (e.g. revision-mode upgrades) and others are not.

### 3.6.5 Backend Technology Alternatives

Functional requirements could be met by a Python-only monolith or a Java-based enterprise stack as well. But in terms of project timeline and focus on integration, the following options were not as appropriate:

- A pure Python full stack would need further trade-offs of the UI ecosystem, and might be slower to iteration on the frontend,
- More elaborate enterprise frameworks would have architecture overhead that would be proportionately less than project scope,
- Premature division into numerous microservices would not add enough reliability to make the DevOps more complicated.

Node.js + Express thus continued to be a pragmatic orchestration layer: lightweight enough to develop quickly, but designed in a way that it could grow in a modular way.

## 3.7 Data Layer Technology Review

### 3.7.1 MongoDB for Flexible Educational Content Storage

As the main datastore, MongoDB was employed [5]. The project gathers the heterogeneous educational objects: the transcripts, the groups of the keywords, the summative objects, the revision artifacts, and the progress records associated with the session. This variability, and incremental schema expansion, are suited to a document model.

### 3.7.2 Mongoose for Schema Discipline in an Evolving System

Mongoose was employed to add schema and model framework to MongoDB [6]. This is crucial since iterative projects have the tendency to lose track in lapses of un-data contracts. Mongoose assists with keeping the model-level clear, and yet flexible document-store.

The commit evidence in revision and progress modules demonstrates pragmatic schema modifications and reliability corrections during the course of time, suggesting dynamism of data-model management instead of non-dynamism of one-time design.

### 3.7.3 Data Modeling Strategy for Learning Artifacts

The data model of AITA does not have to be restricted to the plain transcript storage. It contains associated educational objects like group of keywords, summaries, study sessions and revision objects. This indicates an artifact-based learning process in which the learning process is enhanced at every stage.

This pattern is appropriate to the document-oriented data due to the flexibility of shape of artifacts which may change during the implementation process. To illustrate, metadata of revision and progress indicators can easily be extended without strict migration cycles common in strongly relational schemes. Meanwhile, Mongoose schemas are not so unstructured as to lose data integrity and predictable API contracts [6].

### 3.7.4 Consistency, Indexing, and Query Practicality

Retrieval performance has a strong influence on the perceived responsiveness in the educational systems. The design of AITA focuses on the practical query paths with the data being organized around the relationships of user ownership and study-session. This facilitates the usual activities like displaying the transcripts of a user, browsing related keywords and creation of revision sets based on the preferred sources.

Although this dissertation does not include large-scale benchmarking, the selected model can be used to estimate the predicted workload profile of student access and is consistent with the performance that MongoDB excels in, document access patterns [5].

## 3.8 AI and NLP Technology Review

### 3.8.1 Speech-to-Text and Transcript Processing

AITA includes live speech capture and audio transcription (uploaded) speech capture. It is implemented with a combination of the capabilities of Web Speech API to interact live and OpenAI speech-to-text services to uploaded media [10, 7, 8].

This two-way design is technical reasoned:

- Live speech path supports instant in lecture recording,
- Upload path enhances pragmatism of content recorded and asynchronous learning.

The architecture is thus more robust in the face of actual student use cases rather than being based on one input mode.

### 3.8.2 Generative AI for Correction, Keyword Support, and Study Artifacts

Transcript correction, key word/definition help, summing up generation and chat workflows were done using OpenAI text generation services [9, 7]. This correlates to modern results that the use of LLM-based systems can aid language transformation and assistant educational processes in case of embedding it into well-defined task boundaries [16, 17, 15].

The design in AITA did not focus on training new language models, but coordinating existing APIs in a guided educational pipeline. This methodologically suits an undergraduate systems dissertation based on the value of integration.

### 3.8.3 Model Capability vs. System Responsibility

The main AITA technology review principle is to decoupling model capability and system responsibility. To produce fluent output, language models may rely on the surrounding controls: prompt discipline, context selection, fallback handling and transparent workflow to the user.

This difference is based on the general lessons of LLM engineering (few-shot and instruction-following) literature [16, 17]. The real-world implication of this to AITA is that quality of technology is measured based on end-to-end behavior (helpful transcript correction, reasonable key support, useful revision artifacts), not based on model novelty claims.

### 3.8.4 Speech Technology Practicality and Platform Constraints

The browser speech features coupled with a cloud transcription are an orchestrated resilience choice. Browser APIs might be immediate and can depend on the environment; transcription on the cloud enhances consistency on uploaded media. The two-path approach minimizes single-point dependency, and offers greater user flexibility to various study situations [10, 8].

Patterns of GitHub issue/fixes that dealt with recording behavior and upload handling suggest that this area needed to be continually fine-tuned, which also confirms the necessity of a dual-path architecture, as opposed to a single transcription mode.

### 3.8.5 Python Microservice Role and Pragmatic NLP Choices

The support of analysis was done by another Python service and deployment-oriented changes of the NLTK were noted in the commit history. This illustrates a form of practical tool substitution in the context of platform constraints

(such as depending on heavier dependencies to implement a more reliable deploy, so as to adapt to a proposal), in line with the adaptation logic of proposal-to-implementation in Chapter 2.

## 3.9 Deployment and Operations Technology Review

### 3.9.1 Service Split Across Vercel and Render

The frontend hosting service is based on Vercel, and the backend/Python services are based on Render [14, 13]. This division fits AITA well since it isolates concerns of static/UI delivery and API compute, and is indicative of typical web deployment patterns today.

### 3.9.2 Configuration and Environment Separation

The environment-sensitive configuration practice is also evident in deployment architecture in AITA: frontend and backend have distinct scopes of environment variables, and settings unique to the service are segregated by runtime context. This is significant to both security and maintainability since there should not be any intermixing of credentials and API endpoints across layers.

The deployment and routing fixes in the repository illustrate an actual tightening of this separation over time, in particular towards SPA fallback behavior and cross-service communication.

### 3.9.3 Operational Trade-offs

The split-platform model brings about trade-offs which are significant to record in a technology review:

- Uncoordinated independent service deploys enhance modularity at the cost of endpoint/version coordination,
- Hosted AI solutions enhance functionality, but introduce latency and network-reliability,
- The educational responsiveness is based on the need to strike a balance between AI depth and expediency in response time.

The following trade-offs are not objectionable in the context of AITA and they are in line with the implementation evidence as observed in iterative commits of reliability.

### 3.9.4 Operational Evidence from Integration Fixes

There are several commits that deal with deployment routing, CORS, compatibility with the environment, and service resilience. Such records are significant in a technology review since they indicate that platforms selected did not just pass the test of theory but also, it was practically tested by means of troubleshooting and hardening.

## 3.10 GitHub Evidence of Technology Evolution

A strong technology review should include implementation evidence, not only documentation references. The AITA GitHub timeline shows iterative confirmation of technology fit [11].

Representative examples include:

- **Keyword/NLP evolution**: migration and fixes for analysis dependencies and definition workflows (e.g., NLTK-focused compatibility updates and keyword pipeline stabilization).
- **Reliability hardening**: fixes for race conditions, middleware path correctness, and endpoint behavior in transcription/revision flows.
- **Learning-feature expansion**: integration of flashcards, MCQ mode, mind map generation, and contextual chat access.
- **User-centered iteration**: repeated UI/UX and mobile refinements based on practical testing feedback.
- **Deployment stabilization**: SPA routing fixes, Render/Vercel configuration improvements, and cross-origin adjustments.

To make this traceability explicit, Table 3.1 lists representative commits reviewed during this chapter.

This evidence supports two conclusions:

1. Technology choices were progressively validated by real integration outcomes,
2. Final architecture reflects measured adaptation, not accidental scope drift.

Table 3.1: Representative GitHub evidence for technology evolution

| Commit | Technology relevance |
|---|---|
| `233a5b0` | Migration toward NLTK-based analysis path for deployment compatibility. |
| `bc2ec25` | Python runtime/deployment hardening for Render environment reliability. |
| `438cf72` | GPT-driven keyword extraction and language-aware definition workflow upgrade. |
| `2a7c665` | Addition of revision module architecture (flashcards/MCQ) and route integration. |
| `8dbee5a` | Integration of AI-powered mind map capability into the learning pipeline. |
| `0dfa5f0` | Persistent JWT login and mobile-oriented UX reliability refinements. |
| `7ae7aba` | Full bilingual coverage expansion across frontend pages. |
| `21c5a72` | Multi-bug reliability pass across auth, keywords, transcription, and UX. |

## 3.11 Security and Privacy Technology Review

Since AITA involves user accounts and the personally generated study content, security posture at application level must be included in the technology review. The stack implemented enforces realistic controls by authenticated API, tokenized session flows, verification and reset paths and access to user-scoped content.

This is part of the general web application security advice that identity, access limits and secret management should be considered first-order architectural issues, not an after-thought. On dissertation scope, the controls implemented are appropriate to illustrate intent of secure-by-design, but recognizes that institutional compliance controls auditing falls out of project scope.

Technologically, the JWT-oriented auth and middleware-protected routes are suitable in stateless interactions between APIs in a distributed web deployment. The frequent auth and route hardening history is supported by this evaluation with implementation evidence [11].

## 3.12 Testing and Quality-Engineering Technology Review

AITA testing technology represents a pragmatic mixed approach to automation as opposed to complete formal automation. The Python service consists of script-

based automated checks and the majority of full-stack behavior was checked via integration-oriented scenario testing. This aligns with project restrictions, and the necessity to test end-to-end multi-stage educational workflows.

Technologically speaking, it implies that quality assurance focus is put on the correctness of the user journey: is it possible that, having left the lecture capture, a learner can generate a revision artifact without breaking the workflow? In case of an integration-based project, this requirement is justifiable, but does not supersede overall unit/component/e2e suites that are typically required in production settings.

The most important discovery is that the chosen technologies could be experimented in practical aspects, whereas the future maturity is to have a broader automated coverage and profiling depth of performances.

## 3.13 Synthesis: Objective-to-Technology Alignment

Table 3.2 summarizes how the reviewed stack supports project objectives.

Table 3.2: Objective-to-technology alignment in AITA

| **Objective** | **Primary enabling technologies** |
|---|---|
| O1 (capture/transcription) | Web Speech API + OpenAI Speech-to-Text pipeline [10, 8]. |
| O2 (readability) | LLM-based transcript correction orchestration [9]. |
| O3 (keyword support) | GPT-assisted extraction and definition flow with Python analysis support. |
| O4 (revision artifacts) | Backend generation services for summaries, flashcards, MCQ, and mind map modules. |
| O5 (active recall) | Revision mode interaction design and progress-oriented data structures. |
| O6 (usability) | React/Vite UI architecture, reusable components, bilingual support [1, 2]. |
| O7 (security) | JWT-based auth flow, protected routes, verification and reset account lifecycle. |
| O8 (integration depth) | Node/Express orchestration with MongoDB/Mongoose persistence and cloud deployment split [3, 4, 5, 6, 14, 13]. |

This synthesis confirms that the technology stack is not merely modern in iso-

lation; it is functionally aligned with the dissertation's educational and engineering objectives.

## 3.14 Comparison with Alternative Approaches

Several alternatives were possible, but less suitable under project constraints:

- **Monolithic desktop tool** instead of web stack: reduced accessibility and weaker deployment realism.
- **Single-mode transcription only**: lower usability across varied learner contexts.
- **From-scratch NLP models**: high research overhead and limited practical delivery within timeline.
- **Strict relational-first schema**: less flexible for rapidly evolving educational artifact shapes.

The selected stack therefore represents a balanced engineering choice for breadth, feasibility, and final-year demonstrability.

## 3.15 Technology Limitations and Trade-offs

Despite suitability, the selected technologies introduce known trade-offs:

- AI output variability can affect consistency of corrections and generated revision content.
- External API dependence introduces latency and cost considerations.
- Browser speech behavior differs across environments, requiring defensive UX handling.
- Automated test coverage remains limited relative to production-grade expectations.

These limitations are acknowledged explicitly and carried into Chapter 5 evaluation and Chapter 6 future work.

## 3.16 Why This Technology Stack Is Appropriate for AITA

The reviewed stack is appropriate because it satisfies the project's central implementation demands:

1. Rapid iterative full-stack development,
2. Practical AI-assisted feature integration,
3. Support for heterogeneous educational data,
4. Cloud-deployable architecture,
5. Traceable evolution evidenced by GitHub history.

Therefore, technology selection in AITA is justified not only by documentation-level suitability but also by repository-level implementation evidence.

## 3.17 Chapter Summary

This chapter reviewed and justified the technologies used in AITA through both literature and implementation evidence. React/Vite, Node/Express, MongoDB/Mongoose, OpenAI services, Web Speech API, and Vercel/Render were shown to be coherent with project objectives and iterative delivery constraints. GitHub evolution records further demonstrated that these choices were actively validated and refined in practice.

The next chapter presents the full system design, including architecture, data flow, module relationships, and security-aware implementation structure.

# Chapter 4

# System Design

## 4.1 Chapter Overview

This chapter explains the *how* of AITA: how requirements were transformed into a working architecture, how modules collaborate at runtime, and how design decisions support reliability, usability, and extensibility. The chapter follows a system-design perspective grounded in architectural decomposition and flow consistency.

The design discussion is organized from macro to micro:

1. Overall deployment and component architecture,
2. End-to-end interaction workflows,
3. Domain/data model design,
4. Backend API and service orchestration,
5. Frontend interaction design,
6. Security and operational controls,
7. Representative code and algorithm specifications.

This structure ensures traceability between design intent and implemented modules.

## 4.2 System Architecture Design

### 4.2.1 Architectural Style

AITA is a layered web architecture that is service-oriented. The system is structured in the three main tiers of runtime:

- **Presentation layer**: React/Vite interface to communicate with users [1, 2].
- **Application layer**: Node.js/Express business logic, orchestration and API control backend [3, 4].
- **Information and AI layer**: MongoDB storage and AI/NLP services (OpenAI APIs and Python analysis microservice) [5, 6, 7].

AITA can use this style since both transactional consistency (accounts, stored transcripts, revision history) and probabilistic generation paths (correction, keyword/summary/revision generation) are needed during educational workflows.

### 4.2.2 High-Level Architecture Diagram

Figure 4.1 presents the top-level architecture.

The frontend will be connected to the backend via authenticated REST/SSE APIs as illustrated in Figure 4.1. The backend organizes long-term storage, and AI-reliant functions. Keywords are analyzed using a special Python service, and frontend hosting (Vercel) and API services (Render) are deployed to the cloud [14, 13].

### 4.2.3 Design Rationale for Split Services

The split design takes care of two realities of implementation:

- These runtime characteristics of different workloads include: static UI delivery, request-response API, streaming endpoint, and NLP analysis.
- Isolation of failure: When a single service path fails (e.g. an endpoint of the analysis latency), the entire platform ought not to fall.

This justification can be seen in the iterative hardening of reliability and configuration of deployments in the repository [11].

## 4.3 UML-Oriented Design Views

### 4.3.1 Use-Case View

The functional behavior can be summarized through key actor-use case relationships:

- **Student**: register/sign in, capture/upload lecture, save transcript, view/edit artifacts, generate revision sets, chat with Kiki.

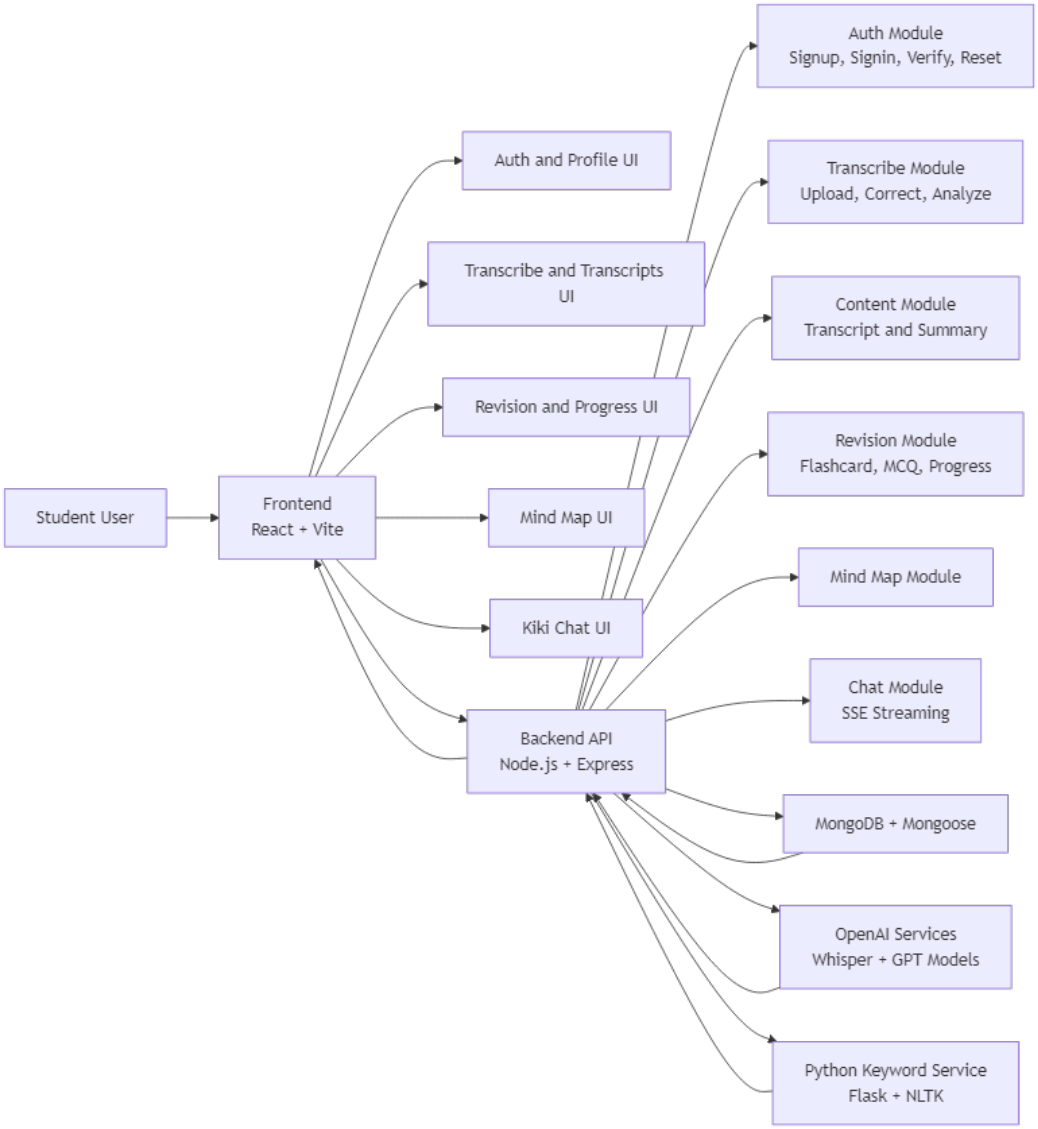


Figure 4.1: AITA high-level system architecture.

- **System**: validate identity, process transcript, invoke AI services, persist outputs, return contextual responses.
- **External services**: OpenAI APIs, browser speech interface, Python analysis service.

This use-case view guided endpoint partitioning and page-level navigation design.

### 4.3.2 Component and Interaction View

Figure 4.2 provides the end-to-end workflow interaction view, equivalent to a runtime sequence perspective from input to revision outputs.

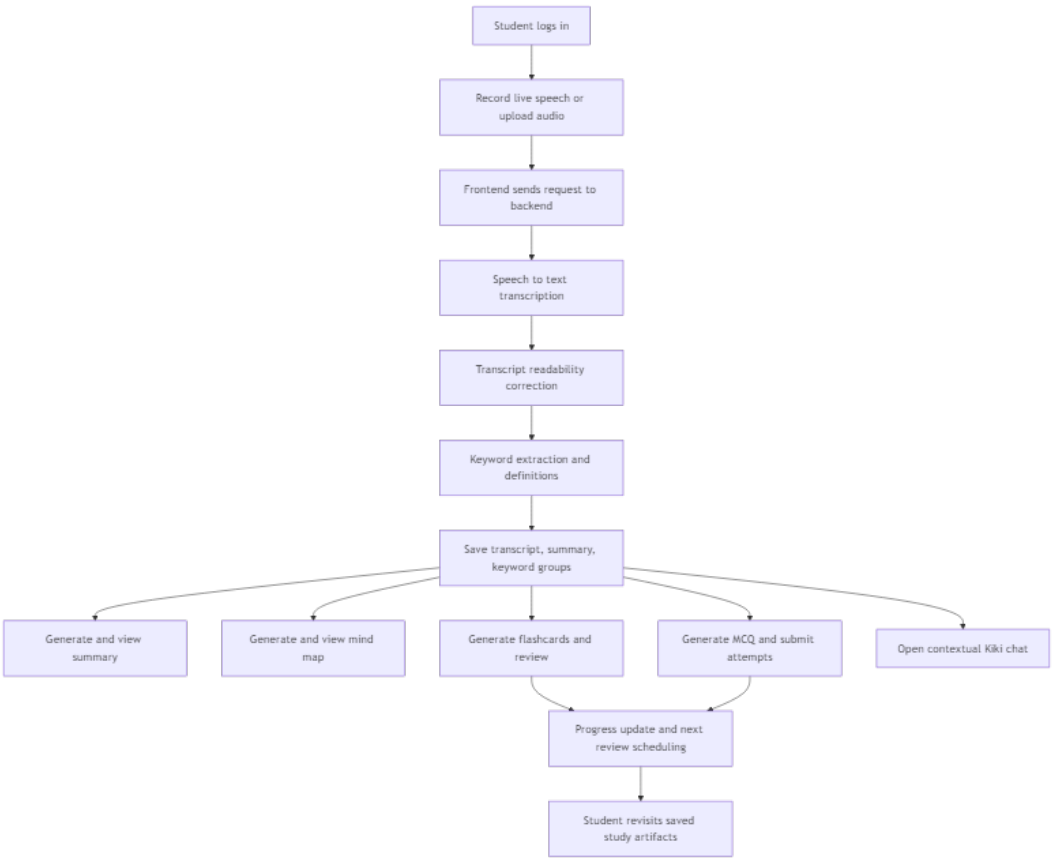


Figure 4.2: End-to-end workflow from lecture ingestion to study outputs.

The interaction view reveals a key design property: AITA is pipeline-centric. Early-stage components (capture/transcription) directly affect downstream quality (keywords, summaries, revision artifacts), so orchestration and fallback handling were prioritized over isolated local optimizations.

### 4.3.3 Domain Model View

Figure 4.3 presents the conceptual domain/data model (UML/ER-style view).

This model was used to enforce user-owned content boundaries and to support multi-artifact learning sessions.

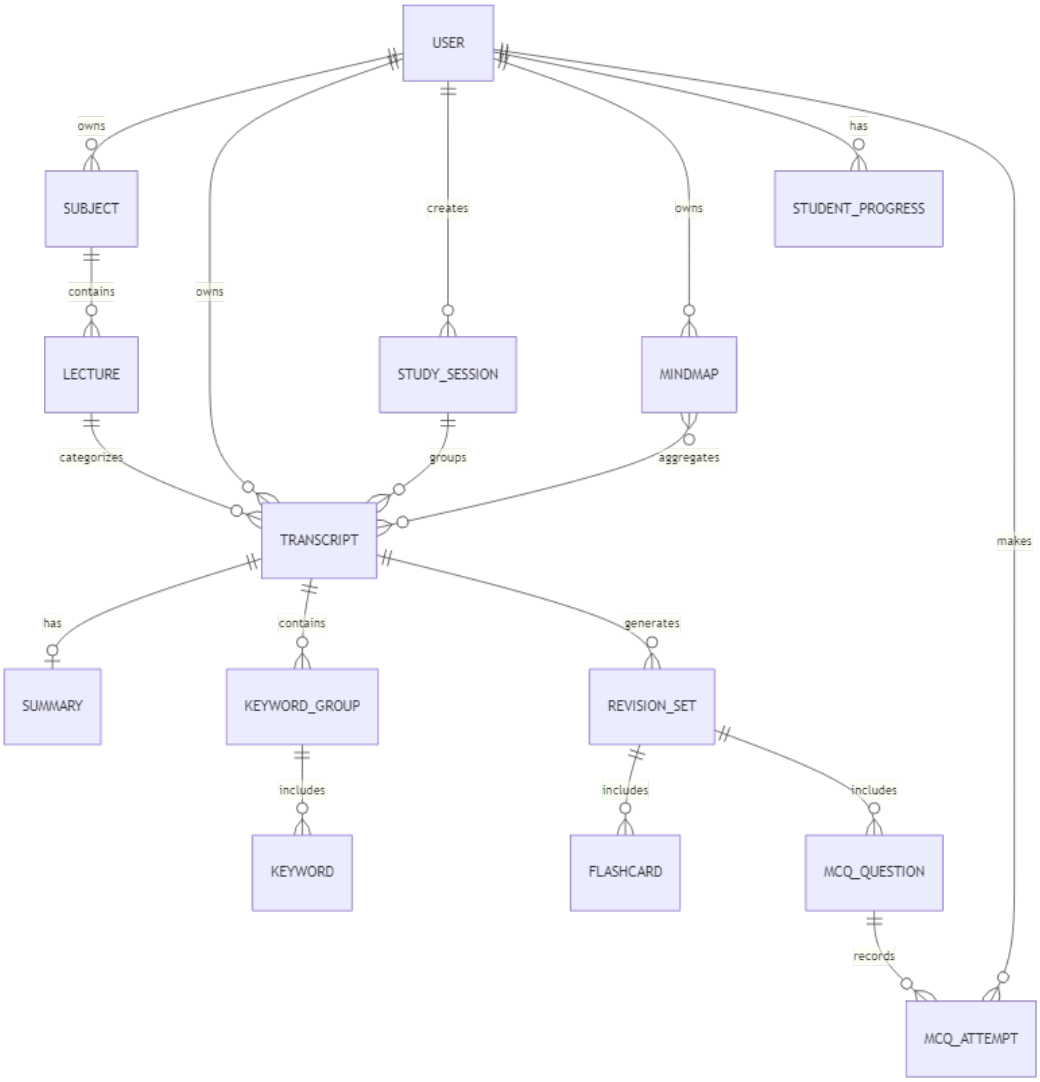


Figure 4.3: Conceptual data model of user, lecture, transcript, and revision artifacts.

## 4.4 Backend System Design

### 4.4.1 Route and Module Decomposition

The backend follows modular route decomposition:

- `/api/auth`: registration, sign-in, verification, reset.
- `/api/transcribe`: upload and transcription-related operations.

- `/api/content`: transcript/content persistence and retrieval.
- `/api/chat`: streaming conversational assistance.
- `/api/mindmaps`: mind map generation and retrieval.
- `/revision`: flashcards, MCQ, progress services.
- `/api/analyze`: proxy to Python analysis service.

This decomposition maps cleanly to user-facing capabilities and minimizes cross-domain coupling.

### 4.4.2 Server Composition Snippet

The snippet below (from implemented backend entry composition) shows route assembly and the analysis proxy boundary:

```
app.use("/api/auth", authRoute);
app.use("/api/transcribe", transcribeRoute);
app.use("/api/users", protectedRoute, userRoute);
app.use("/api/content", contentRoute);
app.use('/revision', revisionRouter);
app.use('/api/chat', chatRoute);
app.use('/api/mindmaps', protectedRoute, mindmapRoute);

app.post('/api/analyze', async (req, res) => {
  // forward transcript to Python backend for keyword analysis
});
```

This structure supports explicit control points for security middleware and service handoff.

### 4.4.3 Streaming Chat Design (SSE)

Kiki AI chat is implemented using Server-Sent Events to stream token chunks progressively. This design improves perceived responsiveness and keeps implementation simpler than bidirectional protocols for this use case.

```
res.setHeader('Content-Type', 'text/event-stream');
res.setHeader('Cache-Control', 'no-cache');
res.setHeader('Connection', 'keep-alive');

// stream chunks from OpenAI response -> frontend
res.write('data: ${JSON.stringify({ text })}\n\n');
```

The SSE path is bounded by context-aware prompt assembly and explicit completion signaling (`[DONE]`), enabling stable frontend rendering.

### 4.4.4 Speech Stream Proxy Design

In the case of real-time capture, the backend has a WebSocket proxy endpoint to stream speech frames and deliver segments of transcripts. The design allows speech transport issues to be separated out of content persistence issues, and makes APIs easier to maintain.

## 4.5 Data Design and Persistence Strategy

### 4.5.1 Core Entities

The system is maintained at the account-level and the learning-artifact level:

- Identity/security: User, Session, verification/reset tokens,
- Artifacts in learning: Transcript, Summary, KeywordGroup/Keyword, MindMap,
- Revision layer: progress structures, flash cards, MCQ questions/attempts.

This isolation enables the management of lifecycle independently, without violating user ownership limits.

### 4.5.2 Design Constraints Applied

Some important persistence limitations are:

- All artifacts are user scoped,
- Cross-artifact cues assist in context reconstruction of learning,
- Optional fields enable non-disruptive migrations to enrich features,
- Retrieval patterns give more importance to frequent actions of students (list, open, revise, regenerate).

The resultant design is flexible and schema disciplined with the help of the Mongoose modeling [6].

## 4.6 Frontend System Design

### 4.6.1 Page and Component Architecture

Frontend is structured in terms of page domains and reusable components:

- Pages for auth and account management,
- Pages concerning transcription and transcripts,
- Revision/mind map/keyword study pages,
- Common elements (sidebar, chat FAB, wrapped protectors of routes).

This hierarchy helps to deliver in modules and enables focused UX changes without regressing the global code.

### 4.6.2 State and Context Design

Mixed state approach is adopted:

- Local state to interact with immediacy of recording (recording state, pending transcript chunks),
- Context providers of cross page continuity (auth, language, chat context).

The design enhances usability since users can move to modules without losing the identity, the language of choice and the continuity of the session.

### 4.6.3 UI Evidence Screenshots

Figures 4.4–4.9 provide implemented UI evidence across core workflows.

## 4.7 Security-Aware Design

### 4.7.1 Authentication and Access Control

The route protection and account lifecycle flows are security controlled:

- Token-based session control,
- User-scoped resource protected routes,
- Verification/reset pathways for account recovery,
- Scoped data retrieval to prevent cross-user leakage.

Figure 4.10 summarizes the implemented auth/security flow.

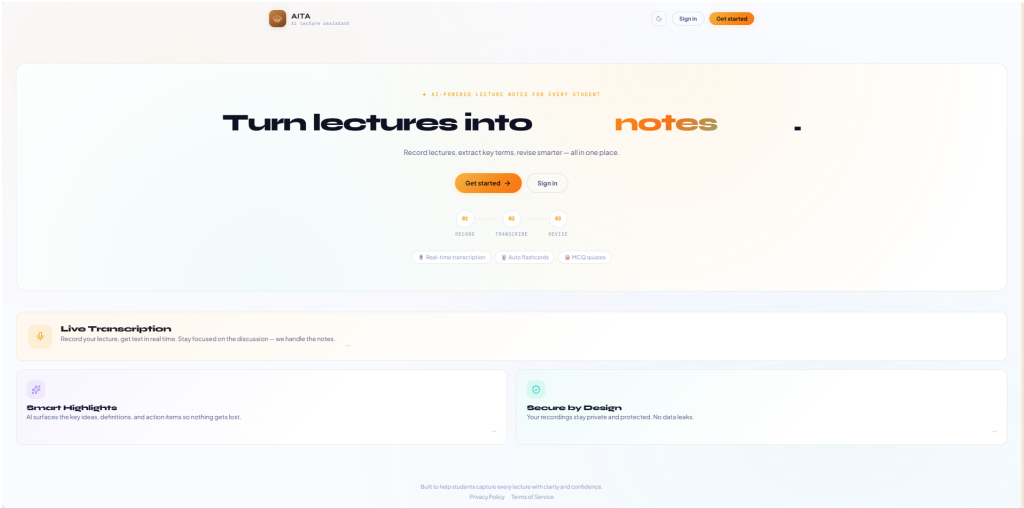


Figure 4.4: Home page interface and entry workflow.

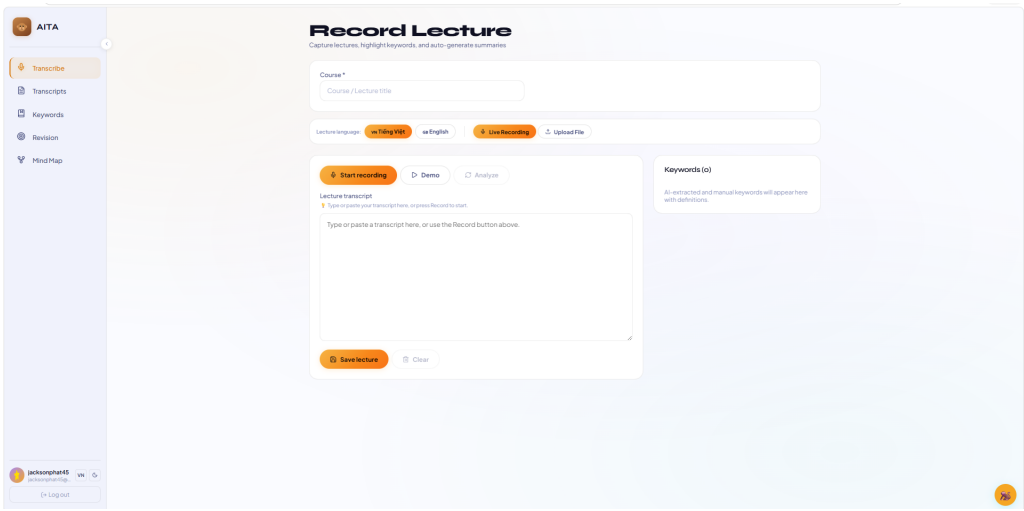


Figure 4.5: Transcription interface (live/upload flow and transcript workspace).

### 4.7.2 Design Trade-offs

The design focuses on the practical application-level security on the scope of the project. It is not purported to be enterprise compliant. This is a clear limit that meets the transparency requirements of methods.

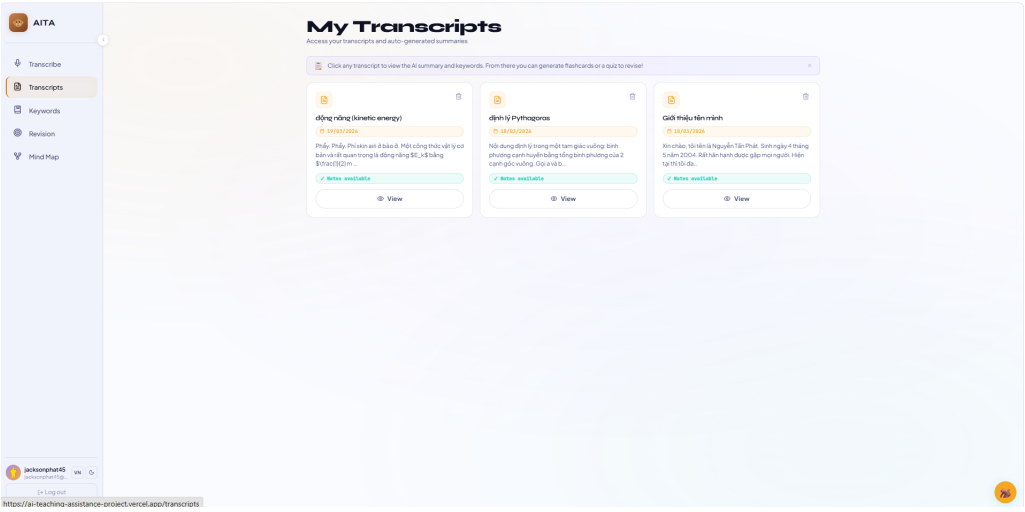


Figure 4.6: Saved transcripts management interface.

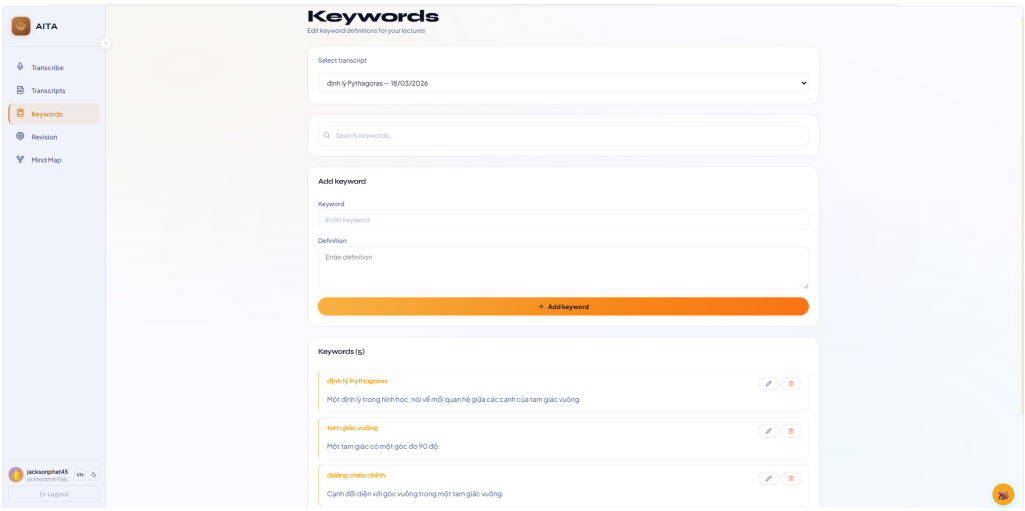


Figure 4.7: Keyword and definition review interface.

## 4.8 Core Algorithms and Processing Logic

### 4.8.1 Algorithm 1: Transcript Processing Pipeline

The logic of the transcript processing can be expressed in the form of the following algorithm:

```
Input:  lecture input (live or uploaded audio), user id, language.
Output: artifacts derived and saved transcript.
```

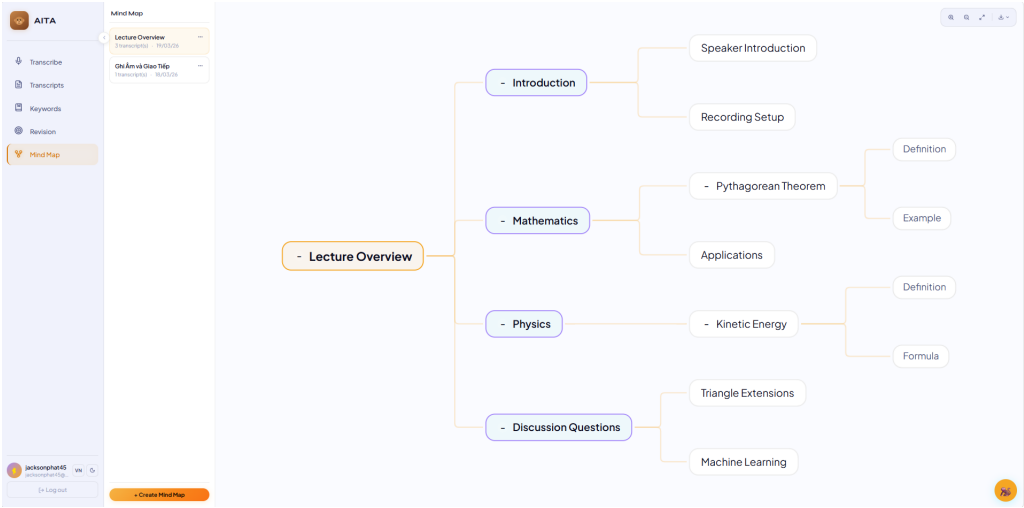


Figure 4.8: Mind map generation and visualization interface.

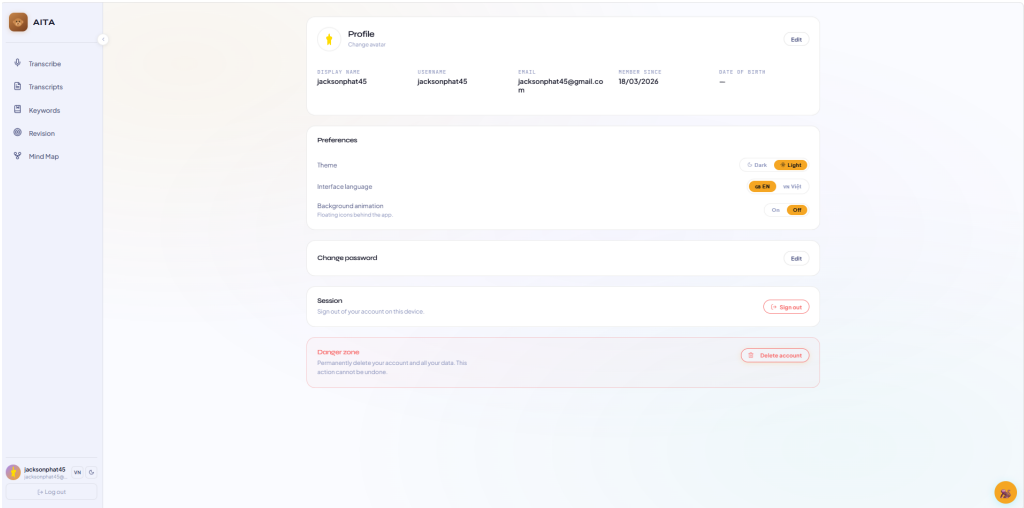


Figure 4.9: Profile and account settings interface.

```
1. Take speech input (live) or get uploaded file.
2. Translate speech to text with pathway of choice.
3. Use AI correction to enhance readability (optionally).
4. Persist transcript under authentication of users.
5. Trigger artifact generation:
   5.1 extract keywords
   5.2 generate definitions
```

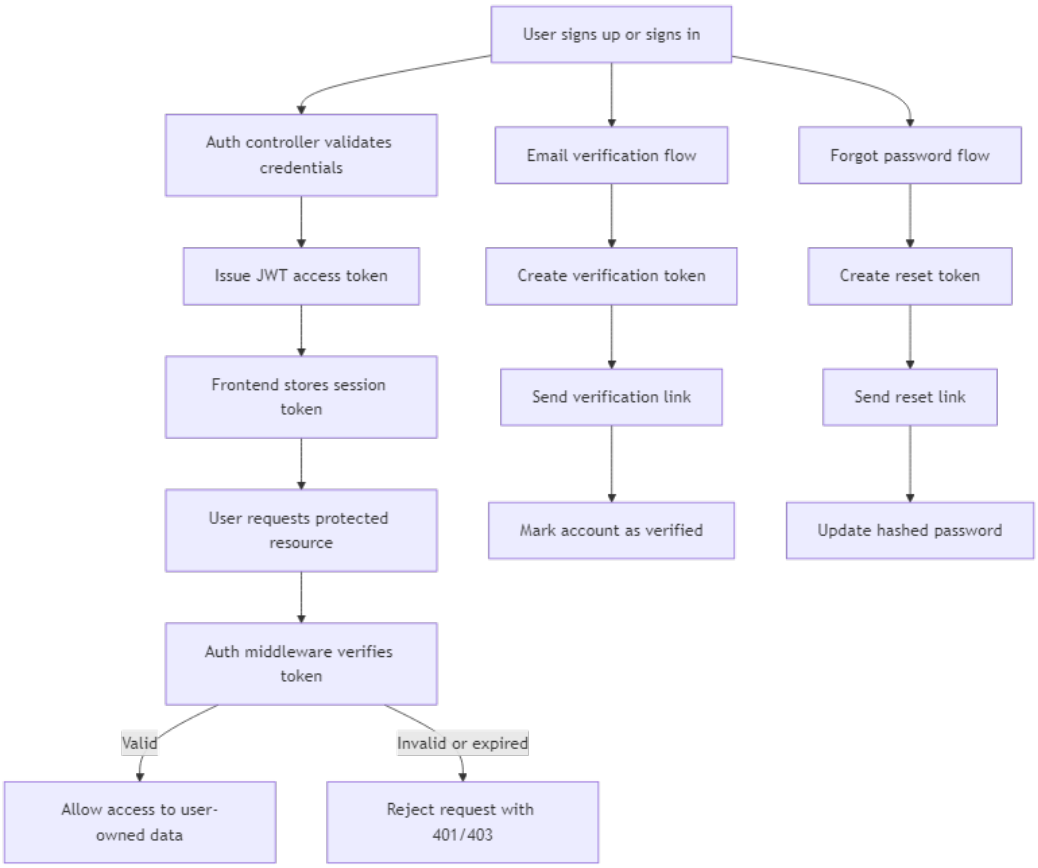


Figure 4.10: Authentication and security flow design.

```
   5.3 generate summary
6. Status of artifact and metadata of returned saved transcripts.
```

This algorithm implements staged enrichment, in which each output can be traced back to the context of transcripts saved.

### 4.8.2 Algorithm 2: Context-Aware Chat Retrieval and Response

```
Input:  user_message, optional lecture context, user identity
Output: streamed assistant response

1. Authenticate message content and user environment.
2. Build system prompt + context block scope.
3. Send request to LLM API (stream mode).
```

```
4. Given a received chunk of tokens:
   4.1 parse content
   4.2 make forward to the client using SSE event.
5. Terminate stream and stream out.
```

The algorithm gives a priority on the context scoping to ensure that the responses remain within learner materials.

## 4.9 Representative Implementation Snippet

A frontend snippet from transcription handling illustrates the three-layer transcript strategy (final corrected text, pending chunk queue, interim speech text):

```
if (finalChunk.trim()) {
  const id = ++chunkIdRef.current;
  pendingChunksRef.current = [...pendingChunksRef.current, { id, raw }];

  if (autoCorrectRef.current) {
    correctTranscript(tokenRef.current, raw.trim(), transcribeLang)
      .then(({ corrected }) => removePending(corrected + ' '))
      .catch(() => removePending(raw));
  } else {
    removePending(raw);
  }
}
```

Design implication: user experience remains responsive while correction runs asynchronously, avoiding UI blocking during speech capture.

## 4.10 Design Decisions and Their Effects

Table 4.1 summarizes key design decisions and observed effects.

## 4.11 Scalability and Extensibility Considerations

The present design does not require a significant replacement of architecture, as it can be extended in small steps:

- Other artifact generators may be added subsequent to transcript persistence,

Table 4.1: Design decision and impact mapping.

| **Design decision** | **Observed implementation impact** |
|---|---|
| Split frontend/backend deployment | Independent release cycles and clearer operational boundaries. |
| Pipeline-oriented artifact generation | Strong objective traceability from transcript to revision outputs. |
| SSE chat streaming | Improved perceived latency and conversational responsiveness. |
| Dual transcription pathway | Better resilience across live and asynchronous study contexts. |
| User-scoped persistence models | Safer content isolation and cleaner retrieval logic. |

- Revision/progress models can be used to add new study analytics,
- Service strategy can be extended when the AI provider gets abstraction,
- Further expansion of multilingual possible via existing i18n structure.

Limitations in automated test depth and high-scale performance benchmarking are known; they are reflected as future work and not as assumptions.

## 4.12 Chapter Summary

In this chapter, the entire system design of AITA was introduced including architecture and UML based views to the module decomposition, data strategy, security controls as well as core processing algorithms. The design illustrates the ability of an implementation based architecture to provide an end-to-end AI-assisted learning workflow and be traceable, modular, and practical to deploy.

The following chapter compares this deployed system with the project goals, such as strengths, limitations that have been witnessed, and real-world testing experiences.

# Chapter 5

# System Evaluation

## 5.1 Chapter Overview

This chapter compares AITA with the objectives as set in Chapter 1 and the validation methodology that is set in Chapter 2. It is all implementation-based evaluation: what the system shows it does, where it is doing, and where it is not doing so.

The chapter has been divided into:

- Assessing plan and sources of evidence,
- Objective-by-objective assessment,
- Representative test results and tables of results,
- Strengths and weaknesses,
- Important reflection on validity and areas of future improvement.

The structure aids in transparent, criteria-based judgement as opposed to promotion description.

## 5.2 Evaluation Strategy

### 5.2.1 Evaluation Basis

The evaluation is in consonance with O1–O8 and mixed evidence:

- Put routes, services and data models into place,
- Integrated user-flow testing, across modules,

- About ten real world feature oriented test cases,
- Repository/deployment data of reliability hardening and defect elimination [11].

Measurement of long-term impact of classroom is not asserted in the chapter. Rather it assesses system-level functionality and usefulness of the platform delivered.

### 5.2.2 Assessment Scale

All the objectives are evaluated on three qualitative levels:

- **Achieved**: objective met with stable evidence implemented.
- **Partially met**: goal has been attained, but with significant constraints.
- **Not met**: goal was not adequately performed to the existing extent.

## 5.3 Objective-Based Evaluation

Table 5.1 summarizes the primary objective-level findings.

Table 5.1 shows that six objectives are achieved and two are partially achieved. No objective is classified as not achieved.

## 5.4 Results from Real-World Feature Testing

To ensure the behavior of the workflow under realistic usage conditions, the project also contained about ten practical feature-oriented test cases. These examples focus on practical and usability testing as opposed to controlled educational tests.

In general, the test set reveals that AITA is functionally consistent when the entire pipeline of lecture-to-revision is used, but still, it has the predicted variability of AI-related outputs and behavior with browsers.

## 5.5 System Strengths

### 5.5.1 Integrated End-to-End Workflow

A key strength is integration depth. AITA does not stop at transcription; it connects capture, correction, term support, summarization, revision generation, and contextual chat in one continuous workflow. This directly addresses workflow friction between lecture attendance and revision preparation.

Table 5.1: Evaluation of AITA against project objectives

| Obj. | Evaluation focus | Status | Evidence summary |
|---|---|---|---|
| O1 | Lecture capture and transcription reliability | Achieved | Live and upload pathways implemented; transcription workflow operational across key scenarios. |
| O2 | Transcript readability improvement | Achieved | AI correction integrated into transcript flow; outputs are visibly more structured for study use. |
| O3 | Keyword and definition support | Achieved | Keyword extraction and definition generation available with fallback-aware service handling. |
| O4 | Revision artifact generation | Achieved | Summaries, flashcards, MCQ, and mind map outputs implemented as reusable study artifacts. |
| O5 | Active-recall workflow support | Partially achieved | Revision mode supports active recall, but longitudinal learning-impact measurement is not yet conducted. |
| O6 | End-to-end usability | Partially achieved | Full workflow is usable and bilingual, but some edge-case UX/performance variability remains. |
| O7 | Account/content security | Partially achieved | Auth and protected routes implemented; formal security audit and compliance certification are out of scope. |
| O8 | Full-stack integration depth | Achieved | Multi-module frontend/backend/AI/data integration is deployed and operationally validated. |

### 5.5.2 Practical Educational Utility

The system generates reusable study objects that are in line with active-recall principles [12]. This, in terms of its evaluation, has an immediate practical value without long-term cohort studies.

### 5.5.3 Implementation Maturity for Final-Year Scope

The project exhibits end-to-end capability: modular frontend/backend structure, persistent data design, orchestration of AI, and deployment on the cloud. The development of repositories is more iterative in nature, than is the development of prototypes [11].

## 5.6 System Weaknesses and Limitations

### 5.6.1 AI Output Variability

Pathways (generative and speech-to-text) are probabilistic. Quality of output may differ based on the audio quality, context clarity and domain terminology. This has an impact on the correction consistency, key word precision and artifacts of downstream revision.

### 5.6.2 Limited Automated Test Depth

Despite the integrated scenario testing, there is still limited end to end automation of the units/components. This slows speed of regression detection with the increase in system complexity.

### 5.6.3 Evaluation Scope Boundary

The assessment establishes viability in functions and integration, but not the causal influence on the learning outcome in the long-term (such as retention gains in a semester). Thus, the claims in pedagogy should be moderate, evidence-based.

### 5.6.4 Operational Constraints

The use of external AI services creates dependencies in terms of latency, availability and costs. Although these constraints can be mitigated (fallback behavior and staged processing) they are still considered part of production-readiness risk.

## 5.7 Critical Reflection

### 5.7.1 What Was Proven

The project demonstrates that it is possible to design an AI-assisted lecture companion as a coherent system, with viable learner-facing value. In particular, it

shows that the O1 to O8 delivery is objective and traceable with no additional objectives being unimplemented.

### 5.7.2 What Was Not Proven

The project fails to demonstrate wide educational scale population-wide effects. It also fails to give in-depth benchmark-type comparisons with other platforms. Such omissions are recognized as research gaps, as opposed to concealed weaknesses.

### 5.7.3 Implications for Future Iteration

Maturity of future evaluation needs to focus on:

- Greater user studies, with outcome measures,
- Increased computerized tests,
- Stronger observability/performance profiling,
- More stringent quality assurance of AI-generated works.

These priorities are based on the profile of the current strengths/limitations directly, and are a practical way to achieve the robustness of production levels.

## 5.8 Threats to Evaluation Validity

### 5.8.1 Internal Validity

Part of the observed improvements is dependent on the composition of workflow instead of the gains of the individual modules, and it is not easy to attribute it. This was alleviated with evaluation based on objectives and cross-module test scenarios.

### 5.8.2 External Validity

The results are best applicable in the context of higher-education where the lecture is the main aspect, as in the case of the project. Sampling needs to be more extensive to be applied to other fields or schools.

### 5.8.3 Construct Validity

Goals like usability, and active-recall support were measured using the implemented workflow behavior, rather than long-term controlled learning experiments. It is a familiar construct boundary and is clearly stated.

## 5.9 Chapter Summary

This chapter compared AITA with its objectives of introduction and discovered that it was doing well in terms of implementation performance in the integrated feature set. The system exhibits strong capabilities in end to end workflow delivery, realistic study support as well as depth of full stack integration. Simultaneously, shortcomings in automated testing profundity, AI steadiness, and protracted educational result data persist.

The assessment hence justifies an equivocal conclusion: AITA is an effective implementation-focused final-year system that has plausible delivered value and a definite next-step research agenda.

Table 5.2: Representative real-world feature test cases

| ID | Scenario | Expected outcome | Result | Notes |
|---|---|---|---|---|
| T1 | User sign-up + email verification | Account creation and verified login success | Pass | Token lifecycle behaved as expected. |
| T2 | Live lecture capture session | Transcript appears continuously with stable updates | Pass | Minor browser-dependent speech variance observed. |
| T3 | Audio file upload transcription | Valid file transcribed and returned for editing | Pass | Supported formats handled under configured limits. |
| T4 | AI transcript correction flow | Corrected text improves readability for saved note | Pass | Quality depends on source audio clarity. |
| T5 | Keyword extraction + definitions | Relevant terms and definitions generated | Pass | Rare domain terms may need manual refinement. |
| T6 | Save transcript + summary generation | Transcript persisted and summary generated | Pass | Asynchronous generation introduces short wait in some cases. |
| T7 | Revision mode (flashcards/MCQ) | Revision artifacts generated and reviewable | Pass | Output quality tied to transcript quality. |
| T8 | Mind map generation | Concept map rendered from selected content | Pass | Complex topics may produce dense visual nodes. |
| T9 | Contextual Kiki chat | Response grounded in selected lecture context | Pass | Streaming response improves perceived responsiveness. |
| T10 | Protected route/content isolation | Unauthorized access blocked for user-scoped data | Pass | No cross-user access observed in tested flows. |

# Chapter 6

# Conclusion

## 6.1 Chapter Overview

This chapter is a conclusion of the dissertation as it summarises the context of the project, redefines the initial goals, and summarises the key results of the system evaluation. It also gives a conclusive reflection on the contribution of the project and practical recommendations towards the further development.

## 6.2 Project Context and Rationale Recap

In this dissertation, the practical educational issue was that in learning environments that are lecture based, students tend to have difficulties in transferring high density information in the classroom to study materials that they can revise. Under traditional workflows, learners will have to sift through transcript-like notes, find terminology, and generate active-recall tools by hand out of the classroom, leading to greater cognitive and time load.

The AITA project was driven by the fact that such a load can be minimized with an all-encompassing AI-enhanced system instead of disjointed systems. The main justification was not to design a new underlying NLP algorithm, but to design a unified end-to-end system in which the entire lecture-to-revision process: capture, correction, comprehension support, artifact generation and contextual study interaction can be supported.

The project had an implementation-based systems approach in terms of methodology. It was suitable in that the technical challenge was more about integrating quality among various interdependent modules (frontend, backend, AI orchestration, persistence, and deployment), as opposed to individual model innovation.

## 6.3 Objectives Recap

In Chapter 1, Chapter 1 established eight main objectives of the project:

1. Give quality lecture capture and transcription services.
2. Enhance the readability of transcripts with the help of AI.
3. Help understand through meaning of keywords and definitions.
4. Create revision resources like summaries, flashcards, MCQs and mind maps.
5. Enabling active-recall directed revision workflow.
6. Provide a complete usable experience of an application.
7. Put into practice effective account/content security measures.
8. Demonstrate depth of integration with end-year project final-year.

These goals were crafted in a manner that they can be assessed and traced based on evidence of implementation so that the conclusions can be made based on the functionality delivered as opposed to claims on proposals which can only be made on proposals.

## 6.4 Summary of Evaluation Findings

Chapter 5 placed AITA in the context of the objective framework and provided a balanced profile of results. The key conclusions are listed below

### 6.4.1 What the Project Successfully Delivered

These results indicated good implementation outcomes in the core:

- The system offers working pathways of lecture capture (live and upload),
- Correction of transcripts and support of key words are merged in the operational processes,
- Revision products (summary, flashcards, MCQ, mind map) are created as study artifacts which can be reused,
- Contextual AI chat is also incorporated in the learning process,

- Full-stack integration is exhibited by implemented multi-service architecture.

Objective-level testing revealed the majority of objectives were met with, and others were partially met with specifically documented constraints. The scope delivered had no objective that was not implemented.

### 6.4.2 What Remains Limited

The boundaries were also defined in the evaluation:

- Input quality and context vary AI output quality,
- The automated test coverage remains low in comparison with production level,
- Testing assures functional/system viability, but is not evidence of long-term causal learning improvements,
- Exogenous dependencies of services bring in latency/cost/availability risk.

These constraints do not nullify the project contribution, instead, they set the boundary of maturity of the current implementation and explain where they need to do work in future.

## 6.5 Overall Contribution

The main contribution of the dissertation is the integration architecture that is delivered and the working educational pipeline. AITA shows that it is possible to integrate the use of modern web engineering with AI-aided processing into a student-facing implementation that can be deployed and decrease the friction between lecture attendance and revision preparation.

Academically, the work makes contributions in three pragmatic aspects:

- **Systems contribution**: a consistent architecture between capture, processing, the generation of artifacts, and interaction.
- **Contribution to implementation**: Full-stack delivery that is objective-traceable and under final-year project constraints.
- **Evaluation contribution**: clear evidence based evaluation with clear strengths, weaknesses and limits of validity.

This is compatible with the implementation-based positioning of the dissertation and the argument that AITA is an implementable final year systems project.

## 6.6 Critical Final Reflection

One of the things that I learned during this project is that educational AI usefulness relies more on how it is designed to work rather than on the capability of the model. Good model APIs do not necessarily produce high performance; usefulness occurs when the generation of output is incorporated in robust, user-scoped and understandable application flows.

Another valuable lesson is methodological: it is necessary to explicitly differentiate between the features implemented and those that are planned in order to make the dissertation credible. The project will keep this boundary during the report and not to over-claim and keep evaluation integrity.

Lastly, the project proves that when clearly recorded, iterative adjustment of proposal assumptions to implementation reality is not a vulnerability. Controlled adaptation may be needed in applied software research, to obtain a stable deployable artifact.

## 6.7 Future Work Directions

The next steps relevant are based on the results of Chapter 5 and they are:

- Increase automated testing (unit, component and end-to-end),
- Carry out bigger user research with quantifiable learning results,
- Improve monitoring of observability and performance during realistic load,
- Provide more quality controls and post-processing of AI-generated work,
- Experiment with wider content consumption and notebook based multi-source study experiences.

These guidelines can be used to expand the existing architecture without having to redesign the entire system.

## 6.8 Final Closing Statement

To sum up, AITA achieves its essence implementation objective: it provides an integrated AI-assisted lecture companion turning the raw lecture input into structured and revision-oriented study output. The project shows impressive full-stack integration depth, real-world educational value, and demonstrates clear methodological traceability.

The dissertation offers a fair and evidence-based perspective on what was and was not accomplished as well as how the system can be improved in future labor, although there are still limitations in empirical scope and robustness on the production level.

# Appendix A

# Source Code Repository

The full source code for the AITA project is available at the following GitHub repository:

`https://github.com/PhatTanNg/AI-Teaching-Assistance-Project`

This repository contains the complete frontend, backend, and Python microservice implementation described in this dissertation.

# Bibliography


[1] React Team. React documentation. https://react.dev/, 2026. Accessed: 2026-04-02.

[2] Vite Team. Vite documentation. https://vitejs.dev/, 2026. Accessed: 2026-04-02.

[3] OpenJS Foundation. Node.js documentation. https://nodejs.org/docs/latest/api/, 2026. Accessed: 2026-04-02.

[4] Express.js Team. Express.js documentation. https://expressjs.com/, 2026. Accessed: 2026-04-02.

[5] MongoDB Inc. Mongodb manual. https://www.mongodb.com/docs/, 2026. Accessed: 2026-04-02.

[6] Mongoose Team. Mongoose documentation. https://mongoosejs.com/docs/, 2026. Accessed: 2026-04-02.

[7] OpenAI. Openai api documentation. https://platform.openai.com/docs, 2026. Accessed: 2026-04-02.

[8] OpenAI. Speech to text guide. https://platform.openai.com/docs/guides/speech-to-text, 2026. Accessed: 2026-04-02.

[9] OpenAI. Text generation and chat completions. https://platform.openai.com/docs/guides/text-generation, 2026. Accessed: 2026-04-02.

[10] MDN Web Docs. Web speech api. https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API, 2026. Accessed: 2026-04-02.

[11] AITA Project Repository. Ai-teaching-assistance-project. https://github.com/PhatTanNg/AI-Teaching-Assistance-Project, 2026. Accessed: 2026-04-02.

[12] P. C. Rivard and P. S. Nugent. Retrieval practice and student learning in higher education: A meta-analytic review. *Educational Psychology Review*, 34(4):2107–2133, 2022.

[13] Render. Render documentation. `https://render.com/docs`, 2026. Accessed: 2026-04-02.

[14] Vercel. Vercel documentation. `https://vercel.com/docs`, 2026. Accessed: 2026-04-02.

[15] A. Zhai, S. Krajewski, and P. A. Brooks. A systematic review of artificial intelligence in education (2010–2020). *Computers and Education: Artificial Intelligence*, 2, 2021.

[16] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pages 1877–1901, 2020.

[17] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pages 27730–27744, 2022.